CS 650 – Full Stack Application Develop

5-1 Short Paper: Primary Versus Secondary Requirements

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

August 4, 2026

FireBoard Training Session Feature Analysis

Missing and Unclear Requirements

The feature request from Captain Mara Estrada is clear about the overall goal but leaves several details that would be important before development begins. We know she wants officers to schedule training sessions, include the training topic, date and time, assign crew members, and allow firefighters to view the daily schedule in FireBoard. However, the request does not explain who is allowed to create or edit training sessions. It also does not say whether only officers can make changes or if other supervisors should have access to them. Another missing detail is whether training sessions can be updated or canceled after they have been created. It is also unclear if firefighters should receive notifications when a new training session is added or changed. These questions would need to be answered before implementation. Clearly identifying stakeholder expectations early helps reduce misunderstandings and improves software quality (Pressman & Maxim, 2020).

If development needed to begin before receiving clarification, several assumptions would have to be made. One reasonable assumption is that only officers or supervisors have permission to create, edit, or delete training sessions, while firefighters can only view them. Another assumption is that training sessions are linked to an existing shift, and only members assigned to that shift can be selected for attendance. It would also be reasonable to assume that each training session has one instructor and takes place at one station. The risk of making assumptions without confirmation is that the completed feature may not match the captain's expectations. If incorrect assumptions are made, the application may require additional redesign and testing later,

increasing both development time and cost. Sommerville (2020) explains that unclear requirements are one of the most common reasons software projects experience delays and additional costs.

Primary and Secondary Requirements

The primary requirements are the features that must exist for the request to be successful. The system must allow an officer to create a training session by entering the training topic, date, time, and assigned crew members. The system must save this information in the database and make it available for firefighters to view. Firefighters should be able to open FireBoard and quickly see what training is scheduled for the current day. Without these core functions, the requested feature would not meet the needs described in the email.

The secondary requirements are supporting features that improve the overall experience but are not absolutely required for the first version. Examples include limiting who can edit training sessions, preventing scheduling conflicts, displaying sessions in chronological order, validating that required fields are completed, and providing confirmation messages after a session is created. These features help improve usability and reliability but are not the main purpose of the request. I identified the primary requirements by focusing on the actions specifically mentioned by Captain Estrada, while the secondary requirements are based on standard software engineering practices that improve system reliability and user experience.

Intended Application Behavior

The intended workflow is simple and matches the captain's request. An officer logs into FireBoard and opens the training section. The officer selects the date and time, enters the training topic, chooses the firefighters who should attend, and saves the session. The application stores information in the database. When firefighters log into FireBoard, they can view the training sessions scheduled for the current day, including the topic, time, and assigned participants. The feature would be considered successful if officers could schedule sessions without errors, and firefighters can easily see accurate training information. Designing applications around user workflows helps ensure that the system supports users' daily tasks effectively (Pressman & Maxim, 2020).

Edge Cases

The application should also handle several edge cases. It should prevent a training session from being saved if required information, such as the topic or date, is missing. The system should stop duplicating or overlapping training sessions for the same crew if conflicts are not allowed. It should also display a helpful message when no training sessions are scheduled for the day instead of showing a blank screen. If a firefighter has not been assigned to a training session, the application should clearly indicate that there are no scheduled trainings for that individual. Handling these situations improves system reliability and reduces user frustration (Sommerville, 2020).

Required Implementation Changes

Database Changes

Supporting this feature requires changes to the database. A new **TrainingSessions** table can be added with fields such as **TrainingID**, **Topic**, **Description**, **Date**, **StartTime**, **EndTime**, **StationID**, **CreatedBy**, and **ShiftID**. Because multiple firefighters may attend the same session, another table called **TrainingParticipants** would connect each training session to the assigned personnel. These tables would work with the existing personnel and shift tables already used by FireBoard. Well-designed database relationships help maintain data integrity and reduce duplicate information (Elmasri & Navathe, 2016).

API Changes

The API would need several new endpoints. A POST endpoint would create a new training session by receiving the topic, date, time, and list of participants. A GET endpoint would return all training sessions for a selected date or shift, so firefighters can view the schedule. A PUT endpoint would update an existing session, while a DELETE endpoint would remove a canceled training session. Each endpoint would return either a success or error response, so the user interface can display appropriate feedback. RESTful APIs provide a standardized way for applications to exchange data between the client and server.

User Interface Changes

The user interface would also require several updates. Officers should have a training scheduling page that contains a form for entering the training details and selecting crew members from a list.

Firefighters should have a daily schedule page that displays upcoming training sessions in an easy-to-read format, including the topic, date, time, and assigned attendees. The interface should clearly indicate when no training sessions are scheduled and provide confirmation after successful updates. A well-designed interface improves usability by making important information easy to find and understand.

How the Proposed Changes Meet the Request

Overall, the proposed changes meet Captain Estrada's request by giving officers a simple way to schedule training sessions while allowing firefighters to quickly view their daily training schedule. The database stores the information, the API makes the data available to the application, and the user interface presents it in a way that is easy to understand. Together, these updates create a practical feature that improves communication, keeps crews informed, and supports the daily operations of the fire district. Following established software engineering principles helps ensure that the feature is maintainable, reliable, and meets stakeholder needs (Pressman & Maxim, 2020).

References

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.

Sommerville, I. (2020). *Software engineering* (10th ed.). Pearson.